



OBJECT ORIENTED PROGRAMMING

FINAL PROJECT

NAME:

- ALI HAMZA (SP25-BCT-004)
- AINA AZHAR (SP24-BDS-003)

SECTION: BDS – 4A & BCT – 2A

SUBMITTED TO: Sir Yasir Munir

Source Code

```
//=====MAIN APPLICATION =====  
public class TypingApp {  
    public static void main(String[] args) {  
        Leaderboard<Result> globalLeaderboard = new Leaderboard<>();  
        new LoginScreen(globalLeaderboard);  
    }  
}  
  
//===== INTERFACES =====  
interface TypingBehavior {  
    void typingMode();  
}  
  
//===== ABSTRACT CLASSES=====  
abstract class User implements TypingBehavior, java.io.Serializable {  
    protected String username;  
    protected String password;  
  
    public User(String username, String password) {  
        this.username = username;  
        this.password = password;  
    }  
  
    public String getUsername() {
```

```

        return username;
    }

    public boolean checkPassword(String input) {
        return password.equals(input);
    }

    public abstract boolean isPremium();
}

// ===== USER CLASSES =====
class FreeUser extends User {
    public FreeUser(String username, String password) {
        super(username, password);
    }

    @Override
    public void typingMode() {
        System.out.println("Free user typing mode activated.");
    }

    @Override
    public boolean isPremium() {
        return false;
    }
}

class PremiumUser extends User {

```

```

public PremiumUser(String username, String password) {
    super(username, password);
}

@Override
public void typingMode() {
    System.out.println("Premium user typing mode activated.");
}

@Override
public boolean isPremium() {
    return true;
}
}

// ===== DATA MODELS
=====

class Result implements java.io.Serializable {
    private String username;
    private double wpm;
    private double accuracy;
    private int mistakes;

    public Result(String username, double wpm, double accuracy, int mistakes) {
        this.username = username;
        this.wpm = wpm;
        this.accuracy = accuracy;
        this.mistakes = mistakes;
    }
}

```

```

    }

    public String getUsername() {
        return username;
    }

    public double getWpm() {
        return wpm;
    }

    public double getAccuracy() {
        return accuracy;
    }

    public int getMistakes() {
        return mistakes;
    }

    @Override
    public String toString() {
        return username + " - WPM: " + wpm + ", Accuracy: " + accuracy + "%" + ", Mistakes: " +
mistakes;
    }
}

// ===== CORE LOGIC CLASSES
=====

class TextGenerator {
    private java.util.ArrayList<String> easy;

```

```

private java.util.ArrayList<String> medium;
private java.util.ArrayList<String> hard;
private String difficultyLevel;

public TextGenerator() {
    easy = new java.util.ArrayList<>();
    medium = new java.util.ArrayList<>();
    hard = new java.util.ArrayList<>();
    difficultyLevel = "easy";
    loadText();
}

public void loadText() {
    try {
        loadFromFile("easy.txt", easy);
        loadFromFile("medium.txt", medium);
        loadFromFile("hard.txt", hard);
    } catch (java.io.IOException e) {
        System.err.println("Error while loading texts: " + e.getMessage());
        if (easy.isEmpty()) easy.add("The quick brown fox jumps over the lazy dog.");
        if (medium.isEmpty()) medium.add("A journey of a thousand miles begins with a
single step. Success is not final, failure is not fatal: it is the courage to continue that
counts.");
        if (hard.isEmpty()) hard.add("The greatest glory in living lies not in never falling, but
in rising every time we fall. The future belongs to those who believe in the beauty of their
dreams.");
    }
}

```

```
public void loadFromFile(String fileName, java.util.ArrayList<String> list) throws  
java.io.IOException {
```

```
    try (java.io.BufferedReader read = new java.io.BufferedReader(new  
java.io.FileReader(fileName))) {
```

```
        String line;
```

```
        while ((line = read.readLine()) != null) {
```

```
            String trimmedLine = line.trim();
```

```
            if (!trimmedLine.isEmpty()) {
```

```
                list.add(trimmedLine);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
public void setDifficulty(int levelNumber) {
```

```
    switch (levelNumber) {
```

```
        case 1: difficultyLevel = "easy"; break;
```

```
        case 2: difficultyLevel = "medium"; break;
```

```
        case 3: difficultyLevel = "hard"; break;
```

```
        default: difficultyLevel = "easy";
```

```
    }
```

```
}
```

```
public String getRandomText() {
```

```
    java.util.ArrayList<String> selectedList;
```

```
    switch (difficultyLevel) {
```

```
        case "medium": selectedList = medium; break;
```

```
        case "hard": selectedList = hard; break;
```

```

        default: selectedList = easy;
    }

    if (!selectedList.isEmpty()) {
        int randomIndex = (int) (Math.random() * selectedList.size());
        return selectedList.get(randomIndex);
    } else {
        return "No text found.";
    }
}

}

class FeedbackAnalyzer {
    private double wpm;
    private double accuracy;
    private int mistakes;

    public void evaluatePerformance(String originalText, String typedText, double
timeTakenSeconds) {
        mistakes = 0;
        int minLength = Math.min(originalText.length(), typedText.length());

        for (int i = 0; i < minLength; i++) {
            if (typedText.charAt(i) != originalText.charAt(i)) {
                mistakes++;
            }
        }
    }
}

```

```

mistakes += Math.abs(originalText.length() - typedText.length());

if (typedText.length() == 0) {
    accuracy = 0.0;
} else {
    accuracy = ((double) (typedText.length() - mistakes) / typedText.length()) * 100;
    if (accuracy < 0) accuracy = 0;
}

if (timeTakenSeconds == 0) {
    wpm = 0.0;
} else {
    double minutes = timeTakenSeconds / 60.0;
    wpm = (typedText.length() / 5.0) / minutes;
}
}

public double getWPM() {
    return wpm;
}

public double getAccuracy() {
    return accuracy;
}

public int getMistakes() {
    return mistakes;
}

```

```
}
```

```
class Leaderboard<T extends Result> {  
    private java.util.ArrayList<T> results;  
    private final String fileName = "results.dat";
```

```
    public Leaderboard() {  
        results = new java.util.ArrayList<>();  
        loadResults();  
    }
```

```
    public void addResult(T result) {  
        results.add(result);  
        saveResults();  
    }
```

```
    public java.util.List<T> getTopResults(int topN) {  
        results.sort((r1, r2) -> Double.compare(r2.getWpm(), r1.getWpm()));  
        return results.subList(0, Math.min(topN, results.size()));  
    }
```

```
    public java.util.List<T> getUserHistory(String username) {  
        java.util.List<T> history = new java.util.ArrayList<>();  
        for (T r : results) {  
            if (r.getUsername().equalsIgnoreCase(username)) {  
                history.add(r);  
            }  
        }  
    }
```

```

        return history;
    }

    private void saveResults() {
        try (java.io.ObjectOutputStream out = new java.io.ObjectOutputStream(new
java.io.FileOutputStream(fileName))) {
            out.writeObject(results);
        } catch (java.io.IOException e) {
            e.printStackTrace();
            System.err.println("Error saving leaderboard results: " + e.getMessage());
        }
    }

    private void loadResults() {
        try (java.io.ObjectInputStream in = new java.io.ObjectInputStream(new
java.io.FileInputStream(fileName))) {
            results = (java.util.ArrayList<T>) in.readObject();
        } catch (java.io.FileNotFoundException e) {
            System.out.println("No existing leaderboard data found. Starting with empty
leaderboard.");
            results = new java.util.ArrayList<>();
        } catch (java.io.IOException | ClassNotFoundException e) {
            e.printStackTrace();
            System.err.println("Error loading leaderboard results: " + e.getMessage());
            results = new java.util.ArrayList<>();
        }
    }
}

```

```
// ===== GUI CLASSES =====

class LoginScreen extends javax.swing.JFrame {

    private javax.swing.JTextField usernameField;
    private javax.swing.JPasswordField passwordField;
    private java.util.ArrayList<User> users = new java.util.ArrayList<>();
    private final String usersFileName = "users.dat";
    private Leaderboard<Result> globalLeaderboard;

    public LoginScreen(Leaderboard<Result> leaderboard) {
        this.globalLeaderboard = leaderboard;
        loadUsers();

        setTitle("Typing App Login");
        setSize(300, 250);
        setDefaultCloseOperation(javax.swing.JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        javax.swing.JPanel panel = new javax.swing.JPanel(new java.awt.GridLayout(0, 1, 5, 5));
        panel.setBorder(javax.swing.BorderFactory.createEmptyBorder(10, 10, 10, 10));

        panel.add(new javax.swing.JLabel("Username:"));
        usernameField = new javax.swing.JTextField();
        panel.add(usernameField);

        panel.add(new javax.swing.JLabel("Password:"));
        passwordField = new javax.swing.JPasswordField();
        panel.add(passwordField);
    }
}
```

```

        javax.swing.JPanel buttonPanel = new javax.swing.JPanel(new
java.awt.FlowLayout(java.awt.FlowLayout.CENTER, 10, 0));

        javax.swing.JButton loginButton = new javax.swing.JButton("Login");
        javax.swing.JButton registerButton = new javax.swing.JButton("Register");
        buttonPanel.add(loginButton);
        buttonPanel.add(registerButton);
        panel.add(buttonPanel);
        add(panel);
        setVisible(true);

        loginButton.addActionListener(e -> login());
        registerButton.addActionListener(e -> register());
    }

    private void login() {
        String uname = usernameField.getText().trim();
        String pass = new String(passwordField.getPassword());

        if (uname.isEmpty() || pass.isEmpty()) {
            javax.swing.JOptionPane.showMessageDialog(this, "Username and password cannot
be empty.", "Login Error", javax.swing.JOptionPane.WARNING_MESSAGE);
            return;
        }

        for (User user : users) {
            if (user.getUsername().equals(uname) && user.checkPassword(pass)) {
                javax.swing.JOptionPane.showMessageDialog(this, "Login successful!");
                dispose();
                new DashboardScreen(user, globalLeaderboard);
            }
        }
    }

```

```

        return;
    }
}

javax.swing.JOptionPane.showMessageDialog(this, "Invalid username or password!",
"Login Failed", javax.swing.JOptionPane.ERROR_MESSAGE);
}

private void register() {
    String uname = usernameField.getText().trim();
    String pass = new String(passwordField.getPassword());

    if (uname.isEmpty() || pass.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Username and password cannot
be empty.", "Registration Error", javax.swing.JOptionPane.WARNING_MESSAGE);
        return;
    }

    for (User user : users) {
        if (user.getUsername().equalsIgnoreCase(uname)) {
            javax.swing.JOptionPane.showMessageDialog(this, "Username already taken.
Please choose another.", "Registration Error",
javax.swing.JOptionPane.WARNING_MESSAGE);
            return;
        }
    }

    String[] options = {"Free", "Premium"};

    int type = javax.swing.JOptionPane.showOptionDialog(this, "Select Account Type:",
"Registration",

```

```

        javax.swing.JOptionPane.DEFAULT_OPTION,
        javax.swing.JOptionPane.INFORMATION_MESSAGE, null, options, options[0]);

    User newUser;
    if (type == 0) {
        newUser = new FreeUser(uname, pass);
    } else {
        newUser = new PremiumUser(uname, pass);
    }

    users.add(newUser);
    saveUsers();

    javax.swing.JOptionPane.showMessageDialog(this, "User " + uname + " registered
successfully as " + options[type] + "!");

    usernameField.setText("");
    passwordField.setText("");
}

private void saveUsers() {
    try (java.io.ObjectOutputStream out = new java.io.ObjectOutputStream(new
java.io.FileOutputStream(usersFileName))) {
        out.writeObject(users);
    } catch (java.io.IOException e) {
        e.printStackTrace();

        javax.swing.JOptionPane.showMessageDialog(this, "Error saving user data.", "Save
Error", javax.swing.JOptionPane.ERROR_MESSAGE);
    }
}

```

```

private void loadUsers() {
    try (java.io.ObjectInputStream in = new java.io.ObjectInputStream(new
java.io.FileInputStream(usersFileName))) {
        users = (java.util.ArrayList<User>) in.readObject();
    } catch (java.io.FileNotFoundException e) {
        System.out.println("No user data found. Starting with empty user list.");
        users = new java.util.ArrayList<>();
    } catch (java.io.IOException | ClassNotFoundException e) {
        e.printStackTrace();
        javax.swing.JOptionPane.showMessageDialog(this, "Error loading user data. Starting
fresh.", "Load Error", javax.swing.JOptionPane.ERROR_MESSAGE);
        users = new java.util.ArrayList<>();
    }
}
}

```

```

class DashboardScreen extends javax.swing.JFrame {
    private User currentUser;
    private Leaderboard<Result> leaderboard;

    public DashboardScreen(User user, Leaderboard<Result> leaderboard) {
        this.currentUser = user;
        this.leaderboard = leaderboard;

        setTitle("Dashboard - " + user.getUsername());
        setSize(400, 250);
        setDefaultCloseOperation(javax.swing.JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }
}

```

```

javax.swing.JPanel panel = new javax.swing.JPanel();
panel.setLayout(new java.awt.GridLayout(0, 1, 10, 10));

javax.swing.JLabel welcomeLabel = new javax.swing.JLabel("Welcome, " +
user.getUsername(), javax.swing.SwingConstants.CENTER);
panel.add(welcomeLabel);

javax.swing.JButton startTest = new javax.swing.JButton("Start Typing Test");
javax.swing.JButton viewLeaderboard = new javax.swing.JButton("View Global
Leaderboard");
javax.swing.JButton viewHistory = new javax.swing.JButton("View My History");
javax.swing.JButton exportHistory = new javax.swing.JButton("Export My History");

panel.add(startTest);
panel.add(viewLeaderboard);
panel.add(viewHistory);

if (currentUser.isPremium()) {
    panel.add(exportHistory);
} else {
    javax.swing.JLabel premiumFeatureLabel = new javax.swing.JLabel("Unlock Export
with Premium!", javax.swing.SwingConstants.CENTER);
    premiumFeatureLabel.setForeground(java.awt.Color.GRAY);
    panel.add(premiumFeatureLabel);
}

add(panel, java.awt.BorderLayout.CENTER);
setVisible(true);

```

```
startTest.addActionListener(new java.awt.event.ActionListener() {  
    @Override  
    public void actionPerformed(java.awt.event.ActionEvent e) {  
        DashboardScreen.this.dispose();  
        new TypingTest(currentUser, leaderboard);  
    }  
});
```

```
viewLeaderboard.addActionListener(new java.awt.event.ActionListener() {  
    @Override  
    public void actionPerformed(java.awt.event.ActionEvent e) {  
        new LeaderboardScreen(leaderboard);  
    }  
});
```

```
viewHistory.addActionListener(new java.awt.event.ActionListener() {  
    @Override  
    public void actionPerformed(java.awt.event.ActionEvent e) {  
        new HistoryScreen(leaderboard, currentUser.getUsername());  
    }  
});
```

```
if (currentUser.isPremium()) {  
    exportHistory.addActionListener(new java.awt.event.ActionListener() {  
        @Override  
        public void actionPerformed(java.awt.event.ActionEvent e) {  
            new ExportScreen(leaderboard, currentUser.getUsername());  
        }  
    }  
}
```

```
    });  
    }  
    }  
}
```

```
class TypingTestGUI {  
    private long startTime;  
    private long endTime;  
    private boolean started = false;  
    private StringBuilder typedText = new StringBuilder();  
    private int currentIndex = 0;  
    private javax.swing.Timer timer;  
    private javax.swing.JLabel timerLabel;  
    private javax.swing.JTextPane textPane;  
  
    public interface TypingTestListener {  
        void onTypingComplete(String typedText, double timeTaken);  
    }  
  
    public void showGUI(String textToType, TypingTestListener listener) {  
        javax.swing.JFrame frame = new javax.swing.JFrame("Typing Test");  
        frame.setDefaultCloseOperation(javax.swing.JFrame.DISPOSE_ON_CLOSE);  
        frame.setSize(800, 400);  
        frame.setLocationRelativeTo(null);  
  
        javax.swing.JLabel instructionLabel = new javax.swing.JLabel("Type the text given  
below and press Enter when done: ");  
        instructionLabel.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
```

```

instructionLabel.setFont(new java.awt.Font("Arial", java.awt.Font.BOLD, 14));

textPane = new javax.swing.JTextPane();
textPane.setEditable(false);
textPane.setFont(new java.awt.Font("Monospaced", java.awt.Font.PLAIN, 24));
textPane.setBackground(new java.awt.Color(240, 240, 240));
javax.swing.text.StyledDocument doc = textPane.getStyledDocument();

javax.swing.text.Style defaultStyle = doc.addStyle("default", null);
javax.swing.text.StyleConstants.setForeground(defaultStyle, java.awt.Color.BLACK);
javax.swing.text.StyleConstants.setBackground(defaultStyle, new java.awt.Color(240,
240, 240));

javax.swing.text.Style correctStyle = doc.addStyle("correct", defaultStyle);
javax.swing.text.StyleConstants.setForeground(correctStyle, java.awt.Color.BLUE);
javax.swing.text.StyleConstants.setBold(correctStyle, true);

javax.swing.text.Style incorrectStyle = doc.addStyle("incorrect", defaultStyle);
javax.swing.text.StyleConstants.setForeground(incorrectStyle, java.awt.Color.RED);
javax.swing.text.StyleConstants.setBold(incorrectStyle, true);

try {
    doc.insertString(doc.getLength(), textToType, defaultStyle);
} catch (javax.swing.text.BadLocationException e) {
    e.printStackTrace();
}

javax.swing.JTextField inputField = new javax.swing.JTextField();

```

```

inputField.setFont(new java.awt.Font("Monospaced", java.awt.Font.PLAIN, 20));
inputField.setBackground(new java.awt.Color(250, 250, 250));
inputField.setBorder(javax.swing.BorderFactory.createCompoundBorder(
    javax.swing.BorderFactory.createLineBorder(java.awt.Color.GRAY, 1),
    javax.swing.BorderFactory.createEmptyBorder(5, 5, 5, 5)));

timerLabel = new javax.swing.JLabel("Time: 0.0s | WPM: 0 | Mistakes: 0",
    javax.swing.SwingConstants.CENTER);

timerLabel.setFont(new java.awt.Font("Arial", java.awt.Font.PLAIN, 16));

inputField.addKeyListener(new java.awt.event.KeyAdapter() {
    @Override
    public void keyPressed(java.awt.event.KeyEvent e) {
        if (!started && !Character.isISOControl(e.getKeyChar()) && e.getKeyCode() !=
            java.awt.event.KeyEvent.VK_SHIFT) {
            started = true;
            startTime = System.currentTimeMillis();

            timer = new javax.swing.Timer(100, new java.awt.event.ActionListener() {
                @Override
                public void actionPerformed(java.awt.event.ActionEvent evt) {
                    endTime = System.currentTimeMillis();
                    double elapsed = (endTime - startTime) / 1000.0;
                    updateStatsDisplay(textToType, typedText.toString(), elapsed);
                }
            });
            timer.start();
        }
    }
});

```

```

if (e.getKeyCode() == java.awt.event.KeyEvent.VK_BACK_SPACE) {
    if (currentIndex > 0) {
        currentIndex--;
        if (typedText.length() > 0) {
            typedText.setLength(typedText.length() - 1);
        }
        try {
            doc.setCharacterAttributes(currentIndex, 1, defaultStyle, true);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
} else if (e.getKeyCode() == java.awt.event.KeyEvent.VK_ENTER) {
    completeTest(textToType, listener, frame);
} else if (Character.isDefined(e.getKeyChar()) && !e.isControlDown() &&
currentIndex < textToType.length()) {
    char userChar = e.getKeyChar();
    char originalChar = textToType.charAt(currentIndex);

    javax.swing.text.Style styleToApply;
    if (userChar == originalChar) {
        styleToApply = correctStyle;
    } else {
        styleToApply = incorrectStyle;
    }

    try {
        doc.setCharacterAttributes(currentIndex, 1, styleToApply, true);
    }
}

```

```

        } catch (Exception ex) {
            ex.printStackTrace();
        }

        typedText.append(userChar);
        currentIndex++;

        if (currentIndex == textToType.length()) {
            completeTest(textToType, listener, frame);
        }
    }
}
});

```

```

frame.setLayout(new java.awt.BorderLayout(10, 10));
frame.add(instructionLabel, java.awt.BorderLayout.NORTH);
frame.add(new javax.swing.JScrollPane(textPane), java.awt.BorderLayout.CENTER);
frame.add(inputField, java.awt.BorderLayout.SOUTH);
frame.add(timerLabel, java.awt.BorderLayout.PAGE_END);

```

```

frame.setVisible(true);
inputField.requestFocusInWindow();
}

```

```

private void updateStatsDisplay(String originalText, String currentTypedText, double
elapsedSeconds) {
    int currentMistakes = 0;
    int minLength = Math.min(originalText.length(), currentTypedText.length());

```

```

    for (int i = 0; i < minLength; i++) {
        if (originalText.charAt(i) != currentTypedText.charAt(i)) {
            currentMistakes++;
        }
    }
    if (currentTypedText.length() > originalText.length()) {
        currentMistakes += (currentTypedText.length() - originalText.length());
    }

    double currentWPM = 0.0;
    if (elapsedSeconds > 0) {
        currentWPM = (currentTypedText.length() / 5.0) / (elapsedSeconds / 60.0);
    }

    timerLabel.setText(String.format("Time: %.1fs | WPM: %.0f | Mistakes: %d",
elapsedSeconds, currentWPM, currentMistakes));
}

private void completeTest(String textToType, TypingTestListener listener,
javax.swing.JFrame frame) {
    if (timer != null) timer.stop();
    endTime = System.currentTimeMillis();
    double timeTaken = (endTime - startTime) / 1000.0;
    frame.dispose();
    listener.onTypingComplete(typedText.toString(), timeTaken);
}
}

class TypingTest implements TypingTestGUI.TypingTestListener {

```

```

private User user;
private TextGenerator textGenerator;
private FeedbackAnalyzer feedbackAnalyzer;
private double totalTime;
private String originalText;
private String typedText;
private Leaderboard<Result> leaderboard;

public TypingTest(User user, Leaderboard<Result> leaderboard) {
    this.user = user;
    this.leaderboard = leaderboard;
    this.textGenerator = new TextGenerator();
    this.feedbackAnalyzer = new FeedbackAnalyzer();
    selectLevelAndGetText();
}

private void selectLevelAndGetText() {
    int level = 1;

    if (user instanceof FreeUser) {
        level = 1;

        javax.swing.JOptionPane.showMessageDialog(null, "Free user: Starting with easy
difficulty.", "Difficulty", javax.swing.JOptionPane.INFORMATION_MESSAGE);
    } else if (user instanceof PremiumUser) {
        String inputStr = javax.swing.JOptionPane.showInputDialog(null,
            "Premium user: Enter the Difficulty level (1:Easy, 2:Medium, 3:Hard):",
            "Select Difficulty", javax.swing.JOptionPane.QUESTION_MESSAGE);
        try {

```

```

        if (inputStr != null && !inputStr.trim().isEmpty()) {
            int chosenLevel = Integer.parseInt(inputStr.trim());
            if (chosenLevel >= 1 && chosenLevel <= 3) {
                level = chosenLevel;
            } else {
                javax.swing.JOptionPane.showMessageDialog(null, "Invalid input. Defaulting to
Easy difficulty (1).", "Invalid Input", javax.swing.JOptionPane.WARNING_MESSAGE);
            }
        } else {
            javax.swing.JOptionPane.showMessageDialog(null, "No input. Defaulting to Easy
difficulty (1).", "No Input", javax.swing.JOptionPane.WARNING_MESSAGE);
        }
    } catch (NumberFormatException e) {
        javax.swing.JOptionPane.showMessageDialog(null, "Invalid input. Please enter a
number (1-3). Defaulting to Easy difficulty (1).", "Invalid Input",
javax.swing.JOptionPane.ERROR_MESSAGE);
    }
}

textGenerator.setDifficulty(level);
getText();
}

public void getText() {
    originalText = textGenerator.getRandomText();
    if (originalText.equals("No text found.")) {
        javax.swing.JOptionPane.showMessageDialog(null, "No text found for the selected
difficulty. Please check text files (easy.txt, medium.txt, hard.txt).", "Error",
javax.swing.JOptionPane.ERROR_MESSAGE);
        new DashboardScreen(user, leaderboard);
    }
    return;
}

```

```

    }
    System.out.println("Original Text: " + originalText);
    startTypingTest();
}

private void startTypingTest() {
    TypingTestGUI gui = new TypingTestGUI();
    gui.showGUI(originalText, this);
}

@Override
public void onTypingComplete(String typedText, double timeTaken) {
    this.typedText = typedText;
    this.totalTime = timeTaken;

    System.out.println("Typed: " + typedText);
    System.out.println("Time: " + String.format("%.2f", totalTime) + " seconds.");

    feedbackAnalyzer.evaluatePerformance(originalText, typedText, totalTime);
    storeResult();
    displayResult();
}

public void storeResult() {
    double wpm = feedbackAnalyzer.getWPM();
    double accuracy = feedbackAnalyzer.getAccuracy();
    int mistakes = feedbackAnalyzer.getMistakes();
    Result result = new Result(user.getUsername(), wpm, accuracy, mistakes);

```

```

        leaderboard.addResult(result);
    }

    public void displayResult() {
        double wpm = feedbackAnalyzer.getWPM();
        double accuracy = feedbackAnalyzer.getAccuracy();
        int mistakes = feedbackAnalyzer.getMistakes();
        new ResultScreen(wpm, accuracy, mistakes, user, leaderboard);
    }
}

class ResultScreen extends javax.swing.JFrame {

    public ResultScreen(double wpm, double accuracy, int mistakes, User user,
        Leaderboard<Result> leaderboard) {

        setTitle("Your Typing Test Results");
        setSize(400, 300);
        setDefaultCloseOperation(javax.swing.JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);

        javax.swing.JPanel panel = new javax.swing.JPanel();
        panel.setLayout(new java.awt.GridLayout(0, 1, 10, 10));
        panel.setBorder(javax.swing.BorderFactory.createEmptyBorder(20, 20, 20, 20));

        javax.swing.JLabel titleLabel = new javax.swing.JLabel("Test Complete!",
        javax.swing.SwingConstants.CENTER);
        titleLabel.setFont(new java.awt.Font("Arial", java.awt.Font.BOLD, 20));
        panel.add(titleLabel);
    }
}

```

```
panel.add(new javax.swing.JLabel("Words Per Minute (WPM): " + String.format("%.2f", wpm), javax.swing.SwingConstants.CENTER));
```

```
panel.add(new javax.swing.JLabel("Accuracy: " + String.format("%.2f", accuracy) + "%", javax.swing.SwingConstants.CENTER));
```

```
panel.add(new javax.swing.JLabel("Mistakes: " + mistakes, javax.swing.SwingConstants.CENTER));
```

```
javax.swing.JButton backToDashboardButton = new javax.swing.JButton("Back to Dashboard");
```

```
panel.add(backToDashboardButton);
```

```
add(panel, java.awt.BorderLayout.CENTER);
```

```
setVisible(true);
```

```
backToDashboardButton.addActionListener(new java.awt.event.ActionListener() {
```

```
    @Override
```

```
    public void actionPerformed(java.awt.event.ActionEvent e) {
```

```
        dispose();
```

```
        new DashboardScreen(user, leaderboard);
```

```
    }
```

```
});
```

```
leaderboard.addResult(new Result(user.getUsername(), wpm, accuracy, mistakes));
```

```
javax.swing.JOptionPane.showMessageDialog(this, "Your result has been saved to the leaderboard!", "Result Saved", javax.swing.JOptionPane.INFORMATION_MESSAGE);
```

```
}
```

```
}
```

```
class LeaderboardScreen extends javax.swing.JFrame {
```

```
    private javax.swing.JTextArea leaderboardArea;
```

```

private Leaderboard<Result> leaderboard;

private static final int TOP_N = 10;

public LeaderboardScreen(Leaderboard<Result> leaderboard) {
    this.leaderboard = leaderboard;
    setTitle("Top Performers Leaderboard");
    setSize(400, 500);
    setDefaultCloseOperation(javax.swing.JFrame.DISPOSE_ON_CLOSE);
    setLocationRelativeTo(null);

    setLayout(new java.awt.BorderLayout());
    leaderboardArea = new javax.swing.JTextArea();
    leaderboardArea.setEditable(false);
    leaderboardArea.setFont(new java.awt.Font("Monospaced", java.awt.Font.PLAIN, 14));

    javax.swing.JScrollPane scrollPane = new javax.swing.JScrollPane(leaderboardArea);
    add(scrollPane, java.awt.BorderLayout.CENTER);
    updateLeaderboardDisplay();
    setVisible(true);
}

private void updateLeaderboardDisplay() {
    java.util.List<Result> topResults = leaderboard.getTopResults(TOP_N);
    StringBuilder sb = new StringBuilder();
    sb.append(String.format("%-5s %-15s %-10s %-10s %-10s%n", "Rank", "Username",
"WPM", "Accuracy", "Mistakes"));
    sb.append("-----\n");
}

```

```

    if (topResults.isEmpty()) {
        sb.append("\nNo results yet. Start typing to get on the leaderboard!\n");
    } else {
        for (int i = 0; i < topResults.size(); i++) {
            Result r = topResults.get(i);
            sb.append(String.format("%-5d %-15s %-10.2f %-10.2f %-10d\n",
                (i + 1), r.getUsername(), r.getWpm(), r.getAccuracy(), r.getMistakes()));
        }
    }
    leaderboardArea.setText(sb.toString());
}
}

```

```

class HistoryScreen extends javax.swing.JFrame {
    private javax.swing.JTextArea historyArea;
    private Leaderboard<Result> leaderboard;
    private String username;

    public HistoryScreen(Leaderboard<Result> leaderboard, String username) {
        this.leaderboard = leaderboard;
        this.username = username;
        setTitle("Typing History for: " + username);
        setSize(400, 500);
        setDefaultCloseOperation(javax.swing.JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);

        setLayout(new java.awt.BorderLayout());
        historyArea = new javax.swing.JTextArea();
    }
}

```

```

historyArea.setEditable(false);
historyArea.setFont(new java.awt.Font("Monospaced", java.awt.Font.PLAIN, 14));

javax.swing.JScrollPane scrollPane = new javax.swing.JScrollPane(historyArea);
add(scrollPane, java.awt.BorderLayout.CENTER);
updateHistoryDisplay();
setVisible(true);
}

private void updateHistoryDisplay() {
    java.util.List<Result> userHistory = leaderboard.getUserHistory(username);
    StringBuilder sb = new StringBuilder();
    sb.append(String.format("%-10s %-10s %-10s\n", "WPM", "Accuracy", "Mistakes"));
    sb.append("-----\n");

    if (userHistory.isEmpty()) {
        sb.append("\nNo typing history found for " + username + ".\n");
    } else {
        userHistory.sort((r1, r2) -> Double.compare(r2.getWpm(), r1.getWpm()));
        for (Result r : userHistory) {
            sb.append(String.format("%-10.2f %-10.2f %-10d\n", r.getWpm(),
r.getAccuracy(), r.getMistakes()));
        }
    }
    historyArea.setText(sb.toString());
}
}

```

```

class ExportScreen extends javax.swing.JFrame {
    private Leaderboard<Result> leaderboard;
    private String username;

    public ExportScreen(Leaderboard<Result> leaderboard, String username) {
        this.leaderboard = leaderboard;
        this.username = username;

        setTitle("Export History for " + username);
        setSize(400, 150);
        setDefaultCloseOperation(javax.swing.JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new java.awt.FlowLayout());

        javax.swing.JLabel infoLabel = new javax.swing.JLabel("Click to export your typing
history to a text file.");
        javax.swing.JButton exportButton = new javax.swing.JButton("Export History (TXT)");

        add(infoLabel);
        add(exportButton);

        exportButton.addActionListener(new java.awt.event.ActionListener() {
            @Override
            public void actionPerformed(java.awt.event.ActionEvent e) {
                exportData();
            }
        });
    }
}

```

```

        setVisible(true);
    }

    private void exportData() {
        javax.swing.JFileChooser fileChooser = new javax.swing.JFileChooser();
        fileChooser.setDialogTitle("Save Export File");
        fileChooser.setSelectedFile(new java.io.File(username + "_history.txt"));

        int userSelection = fileChooser.showSaveDialog(this);

        if (userSelection == javax.swing.JFileChooser.APPROVE_OPTION) {
            java.io.File fileToSave = fileChooser.getSelectedFile();
            try (java.io.FileWriter writer = new java.io.FileWriter(fileToSave)) {
                java.util.List<Result> userHistory = leaderboard.getUserHistory(username);

                writer.append(String.format("%s's Typing History\n", username));
                writer.append("-----\n");
                writer.append(String.format("%-10s %-10s %-10s\n", "WPM", "Accuracy",
" Mistakes"));
                writer.append("-----\n");

                if (userHistory.isEmpty()) {
                    writer.append("No typing history found.\n");
                } else {
                    userHistory.sort((r1, r2) -> Double.compare(r2.getWpm(), r1.getWpm()));
                    for (Result r : userHistory) {
                        writer.append(String.format("%-10.2f %-10.2f %-10d\n", r.getWpm(),
r.getAccuracy(), r.getMistakes()));
                    }
                }
            }
        }
    }

```

```
    }

    javax.swing.JOptionPane.showMessageDialog(this, "History exported successfully
to:\n" + fileToSave.getAbsolutePath(), "Export Complete",
javax.swing.JOptionPane.INFORMATION_MESSAGE);

    dispose();
} catch (java.io.IOException ex) {
    javax.swing.JOptionPane.showMessageDialog(this, "Error exporting file: " +
ex.getMessage(), "Export Error", javax.swing.JOptionPane.ERROR_MESSAGE);
    ex.printStackTrace();
}
}
}
}
```